

Lab 4 — LLM / Claude Evidence

Tic-Tac-Toe Bug Fixes and Feature Implementation

Student: Riya Saju

Project: 09_tic_tac_toe

Evidence: Four Claude prompts used during implementation

Claude Chat History

<https://claude.ai/share/6ee268b5-7f22-4623-82b9-11a1ff89cd96>

Prompt 1 — Diagonal win detection

rules.py
24 lines
PY

I'm fixing a Tic-Tac-Toe game. Can you check this file for the win-detection bug? Diagonal wins aren't being detected. Please fix only that issue, keep the existing structure, and briefly tell me what you changed.

11 hours ago

Diagonal lines were never added to `lines`, as the existing `NOTE` comment hints. The fix is two appended lines in `check_winner`, with the rest of the structure untouched.

```
python
def check_winner(board):
    lines = []
    for r in range(3):
        lines.append([board[r][0], board[r][1], board[r][2]])
    for c in range(3):
        lines.append([board[0][c], board[1][c], board[2][c]])
    lines.append([board[0][0], board[1][1], board[2][2]]) # main diagonal
    lines.append([board[0][2], board[1][1], board[2][0]]) # anti-diagonal

    for line in lines:
        if line[0] is not None and line[0] == line[1] == line[2]:
            return line[0]
    return None
```

What changed:

- Replaced the `# NOTE: diagonals are not included here` comment with the two diagonal lines (top-left to bottom-right, and top-right to bottom-left).
- The existing win check loop handles them automatically, since it already works on any list of three cells.
- `is_board_full` is unchanged.

Prompt 2 — Remaining game logic bugs

game_engine.p
y
78 lines

PY

Now lets fix the remaining game logic issues in this file. A win on the final square can be treated as a draw, the game still accepts clicks after a round ends, and occupied cells can be overwritten. Please fix all three issues, keep the existing structure, and don't add any new features. Briefly explain what you change

10 hours ago   

Fixing three issues in a tic-tac-toe game file.

Here are the updated `handle_click` and `check_round_end`. Everything else in the file is unchanged.

python



```
def handle_click(self, pos):
    if self.round_over:
        return # round is finished - ignore clicks until reset
    if self.current_player != HUMAN_SYMBOL:
        return # not your turn - the computer is about to move (or already has)
    cell = board_pos_to_cell(pos)
    if cell is None:
        return
    row, col = cell
    if self.board[row][col] is not None:
        return # cell already occupied
    self.board[row][col] = self.current_player
    self.check_round_end()
    self.current_player = 'O' if self.current_player == 'X' else 'X'
    self._maybe_take_computer_turn()
```

python



```
def check_round_end(self):
    winner = check_winner(self.board)
    if winner:
        self.round_over = True
        self.winner = winner
        return
    if is_board_full(self.board):
        self.round_over = True
```



```
row, col = cell
if self.board[row][col] is not None:
    return # cell already occupied
self.board[row][col] = self.current_player
self.check_round_end()
self.current_player = 'O' if self.current_player == 'X' else 'X'
self._maybe_take_computer_turn()
```

python

```
def check_round_end(self):
    winner = check_winner(self.board)
    if winner:
        self.round_over = True
        self.winner = winner
        return
    if is_board_full(self.board):
        self.round_over = True
        self.winner = None
```

What changed:

- **Win on the last square:** `check_round_end` now checks for a winner first and only falls back to the full-board draw check if there isn't one. Before, a full board was always called a draw, even when the last move completed a line.
- **Clicks after the round ends:** `handle_click` returns early when `self.round_over` is true, so the board is frozen until the player presses R.
- **Overwriting occupied cells:** `handle_click` returns early if the clicked cell isn't `None`. Because it returns before the turn switches, a click on a taken cell doesn't cost you your turn.

I also removed the two `# BUG` comments, since those issues are fixed. The computer's move path needed no changes: it already stops when the round is over and only picks from empty cells.

📄 🔊 🍌 🗨️ ↻ 10 hours ago

Prompt 3 — Persistent scoreboard

game_engine.py
y
82 lines

PY

renderer.py
58 lines

PY

Let's implement the persistent scoreboard from Task 2. Track X wins, O wins, and draws and display them on the game screen. The scores should increase once when a round ends, stay unchanged when starting a new round, and only reset when the whole match is reset. Please keep the existing game logic and UI structure and don't implement Task 4 yet. Briefly explain what you changed.

10 hours ago ↻ ✎ 📄

Designing scoreboard logic with separate round and match resets.

Here are the changes to `game_engine.py`. Everything not shown is unchanged, and `renderer.py` needed no changes.

python



```
def __init__(self):
    self.scores = {'X': 0, 'O': 0, 'draw': 0}
    self._reset_round()

def _reset_round(self):
    self.board = [[None] * 3 for _ in range(3)]
    self.current_player = 'X'
    self.round_over = False
    self.winner = None # 'X', 'O', or None (meaning draw, only valid when round_
```



python



```
def handle_keydown(self, key):
    import pygame
    if key == pygame.K_r:
        self._reset_round()
```

python



```
def check_round_end(self):
    winner = check_winner(self.board)
    if winner:
        self.round_over = True
        self.winner = winner
        self.scores[winner] += 1
    return
if is board full(self.board):
```



```

    return

    if is_board_full(self.board):
        self.round_over = True
        self.winner = None
        self.scores['draw'] += 1

```

python

```

def draw(self, surface, font):
    from game import renderer
    renderer.draw_board(surface, self.board)
    turn_label = "Your turn (X)" if self.current_player == HUMAN_SYMBOL else "Comp
    renderer.draw_text(surface, font, turn_label, (10, 20))
    score_label = f"X: {self.scores['X']}  O: {self.scores['O']}  Draws: {self.s
    renderer.draw_text(surface, font, score_label, (10, 60))

    if self.round_over:
        text = f"{self.winner} wins!" if self.winner else "Draw!"
        renderer.draw_banner(surface, font, f"{text} Press R for a new round.")

```

What changed:

- **Score storage:** `__init__` now creates a `scores` dict (`X`, `O`, `draw`) and then sets up the first round.
- **Splitting out the round reset:** The board, turn, and round-over state moved into `_reset_round()`. Previously, pressing R called `self.__init__()`, which would now wipe the scores along with the board. R calls `_reset_round()` instead, so a new round keeps the scores.
- **Counting once per round:** `check_round_end` increments the winner's score, or the draw count, at the moment it marks the round over. A round can only end once, because clicks and the computer's move are both blocked afterward, so each result is counted exactly once.
- **Display:** `draw` shows a score line under the turn label, using the existing `renderer.draw_text`. It sits above the board, which starts at y=100.

I didn't add a match-reset control, since the separate reset controls belong to Task 4. Until that exists, nothing zeroes the scores except restarting the program. When you get to Task 4, the match reset can be a `self.scores = {...}` reset plus `_reset_round()`.

Prompt 4 — First-player and restart controls

game_engine.py
94 lines

PY

renderer.py
58 lines

PY

main.py
43 lines

PY

Now implement the final task: let the player choose whether X or O starts, and add separate Restart Round and Reset Match controls. If O starts, the computer should automatically make the first move. Restart Round should clear only the board and keep the scoreboard; Reset Match should clear both the board and scoreboard. The selected starting player should apply to the next round. Please preserve all the existing fixes and scoreboard, keep the current structure/UI style, and don't add anything beyond this task. Briefly explain the changes.

10 hours ago

Finalizing the reset and restart logic for board state. >

Here are the changes to `game_engine.py`. Everything not shown is unchanged, and `renderer.py` and `main.py` need no changes.

python

```
def __init__(self):
    self.scores = {'X': 0, 'O': 0, 'draw': 0}
    self.first_player = HUMAN_SYMBOL # who starts the next round
    self._reset_round()

def _reset_round(self):
    self.board = [[None] * 3 for _ in range(3)]
    self.current_player = self.first_player
    self.round_over = False
    self.winner = None # 'X', 'O', or None (meaning draw, only valid when round_
    self._maybe_take_computer_turn() # if O starts, the computer opens the round
```

python

```
def handle_keydown(self, key):
    import pygame
    if key == pygame.K_x:
        self.first_player = HUMAN_SYMBOL # applies from the next round
    elif key == pygame.K_o:
        self.first_player = COMPUTER_SYMBOL # applies from the next round
    elif key == pygame.K_r:
        self._reset_round() # Restart Round: board only
    elif key == pygame.K_m:
        self.scores = {'X': 0, 'O': 0, 'draw': 0} # Reset Match: scores + board
        self._reset_round()
```

python



```
def draw(self, surface, font):
    from game import renderer
    renderer.draw_board(surface, self.board)
    turn_label = "Your turn (X)" if self.current_player == HUMAN_SYMBOL else "Comp
    renderer.draw_text(surface, font, turn_label, (10, 20))
    renderer.draw_text(surface, font, f"Start: {self.first_player}", (290, 20))
    score_label = f"X: {self.scores['X']}   O: {self.scores['O']}   Draws: {self.s
    renderer.draw_text(surface, font, score_label, (10, 60))
    renderer.draw_text(surface, font, "X/O: first R: round M: match", (10, 462))

    if self.round_over:
        text = f"{self.winner} wins!" if self.winner else "Draw!"
        renderer.draw_banner(surface, font, f"{text} Press R for a new round.")
```

What changed:

- **Starting player:** A new `first_player` attribute holds the choice. Pressing **X** or **O** sets it, and it's shown at the top right as "Start: X/O". Because it's only read when a round is set up, changing it mid-round doesn't affect the current round. It applies the next time you restart the round or reset the match.
- **Computer opens as O:** `_reset_round` now sets `current_player` from `first_player`, then calls the existing `_maybe_take_computer_turn()`. That method does nothing when X starts, and plays the first move when O starts, so no new AI logic was needed.
- **Restart Round (R):** This is the same as before: it clears the board and keeps the scores, and now also uses the selected starter.
- **Reset Match (M):** It zeroes the scores and then calls `_reset_round()`, so the board is cleared too. It keeps your starter choice, since that's a setting rather than match progress.
- **Controls hint:** A small line under the board lists the keys, matching the existing "Press R" keyboard style.

The fixes and the scoreboard are untouched. The uploaded `draw` had the `if self.round_over:` banner block duplicated at the end, which drew the banner twice. I removed the duplicate. That wasn't in the version I gave you earlier, so it was probably a copy/paste slip when merging.

10 hours ago